

University Institute of Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Subject Name: Software Engineering

Subject Code: 23CST-206

Software Project Management

Er. Jagmeet Kaur

E8387

ASSISTANT PROFESSOR

CSE

DISCOVER . **LEARN** . EMPOWER

Chapter-4

Software Project Management

- Software configuration management

Software Configuration Management

- When we develop software, the product (software) undergoes many changes in their maintenance phase; we need to handle these changes effectively.
- Several individuals (programs) works together to achieve these common goals. This individual produces several work product (SC Items) e.g., Intermediate version of modules or test data used during debugging, parts of the final product.
- The elements that comprise all information produced as a part of the software process are collectively called a software configuration.

Software Configuration Management

- In Software Engineering, **Software Configuration Management(SCM)** is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle.
- The primary goal is to increase productivity with minimal mistakes. SCM is part of cross-disciplinary field of configuration management and it can accurately determine who made which revision.

Software Configuration Management

- As software development progresses, the number of Software Configuration elements grow rapidly.
- These are handled and controlled by SCM. This is where we require software configuration management.

Therefore, SCM is the discipline which

- Identify change
- Monitor and control change
- Ensure the proper implementation of change made to the item.
- Auditing and reporting on the change made.

Software Configuration Management

- Configuration Management (CM) is a technique of identifying, organizing, and controlling modification to software being built by a programming team.
- **The objective is to maximize productivity by minimizing mistakes (errors).**
- CM is used to essential due to the inventory management, library management, and updation management of the items essential for the project

Why do we need Configuration Management?

- Multiple people are working on software which is consistently updating.
- It may be a method where multiple version, branches, authors are involved in a software project, and the team is geographically distributed and works concurrently.
- It changes in user requirements, and policy, budget, schedules need to be accommodated.

Importance of SCM

- It is practical in controlling and managing the access to various SCIs e.g., by preventing the two members of a team for checking out the same component for modification at the same time.
- **It provides the tool to ensure that changes are being properly implemented.**
- It has the capability of describing and storing the various constituent of software.
- SCM is used in keeping a system in a consistent state by automatically producing derived version upon modification of the same component.

Processes involved in SCM

Configuration Management (CM) ensures that software components and changes are managed systematically. It helps maintain integrity, control, and consistency throughout the software development lifecycle (SDLC). The CM process consists of five main steps:

1. **Configuration Planning & Management** → Lays the foundation by defining policies, roles, and tools.
2. **Configuration Identification** → Defines configuration items (CIs) and establishes baselines.
3. **Configuration Control** → Manages changes, approvals, and version control.
4. **Configuration Status Accounting** → Tracks and reports changes, maintaining historical records.
5. **Configuration Auditing** → Ensures the software matches planned configurations and meets compliance standards.

1. Configuration Planning and Management

This is the strategic aspect of CM, ensuring that processes, tools, and policies are properly defined and followed.

Key Activities:

- **Defining the CM Plan:**
 - Establish roles, responsibilities, and policies for managing configurations.
 - Define tools and techniques for version control, documentation, and auditing.
- **Resource Allocation:**
 - Assign teams and tools to support configuration activities.
- **Integration with Software Development Lifecycle (SDLC):**
 - Ensure that CM is applied throughout all phases, from requirements gathering to deployment.

Example:

- A company establishes a configuration management policy requiring all code changes to be tracked in Git and reviewed before merging into the main branch.



2. Configuration Identification

This is the foundation of CM, where all software items are identified, documented, and categorized.

Key Activities:

- **Identifying Configuration Items (CIs):**
 - CIs include source code files, design documents, test cases, libraries, user manuals, etc.
 - Every item must be uniquely identified for tracking.
- **Defining Baselines:**
 - A baseline is a stable version of a system or software component that serves as a reference point.
 - Baselines help in comparing changes over time.
- **Version Control and Labeling:**
 - Assign version numbers (e.g., v1.0, v1.1) or labels to different configurations.
 - Tools like Git, SVN, and Mercurial are used to track versions.
- **Example:**
 - In a software project, version "1.0" of an application is released as a baseline. Any future updates, such as "1.1" or "2.0," are managed as new configurations with recorded changes.

3. Configuration Control

Configuration Control ensures that all changes to configuration items are reviewed, approved, and implemented in a controlled manner.

Key Activities:

- **Change Request and Review:**
 - Any changes to the software (bug fixes, feature updates, enhancements) must go through a formal change request process.
 - A **Change Control Board (CCB)** evaluates each request based on its impact, feasibility, and risk.
- **Approval and Implementation of Changes:**
 - Only approved changes are implemented.
 - Changes are documented with details like rationale, implementation steps, and test results.
- **Version Control and Branching:**
 - Maintain different branches for development, testing, and production.
 - Use **Git workflows** (e.g., feature branches, release branches) to manage updates systematically.

Example:

- A developer requests a change to improve login security. The CCB reviews the change, approves it, and assigns a version number before implementing it in the next software release.

4. Configuration Status Accounting

This process involves tracking and reporting the status of all configuration items, changes, and versions.

Key Activities:

- **Recording and Tracking Changes:**
 - Maintain logs of every change made to the system.
 - Keep records of when, why, and by whom changes were made.
- **Maintaining Historical Data:**
 - Store previous versions and configurations to allow rollback if needed.
 - Generate reports on software status.
- **Providing Audit and Compliance Reports:**
 - Track whether changes comply with organizational or regulatory requirements.
 - Ensure proper documentation for audits.

Example:

- If a project manager wants to see what changes were made in version 2.0, they can access the configuration status report that lists all modifications since version 1.0.

5. Configuration Auditing

Configuration auditing ensures that the software conforms to defined requirements and that unauthorized changes do not occur.

Key Activities:

- **Functional Audits:**

- Verify that the software performs as intended after changes.
- Check if all approved modifications are correctly implemented.

- **Physical Audits:**

- Ensure that all physical components (e.g., documents, code, libraries) match their defined configurations.

- **Verification of Compliance:**

- Ensure that the software meets standards such as ISO 9001, CMMI, or internal guidelines.
- Detect unauthorized changes and rollback if necessary.

Example:

- Before releasing version 3.0, an audit confirms that all requested features are implemented and no unauthorized changes exist.

References

- <https://www.guru99.com/software-configuration-management-tutorial.html>
- <https://www.geeksforgeeks.org/software-engineering-system-configuration-management/>
- <https://www.javatpoint.com/software-configuration-management>



THANK YOU